GigaPet JavaScript (DevTools Edition) - Implementations & Screenshots

All functions mentioned in this document can be found in script.js.

All buttons and descriptions mentioned are in the DevTools Demo Panel section of index.html and its corresponding webpage.

1 Message Logging

The buttons and relevant instructions mentioned below are all under the **Message Logging** section of the DevTools Demo Panel.

All resulting logs can be seen in the Console Panel of the DevTools.

1.1 Log Info

Log Info Message implementation is done through the *demoLogInfo()* function.

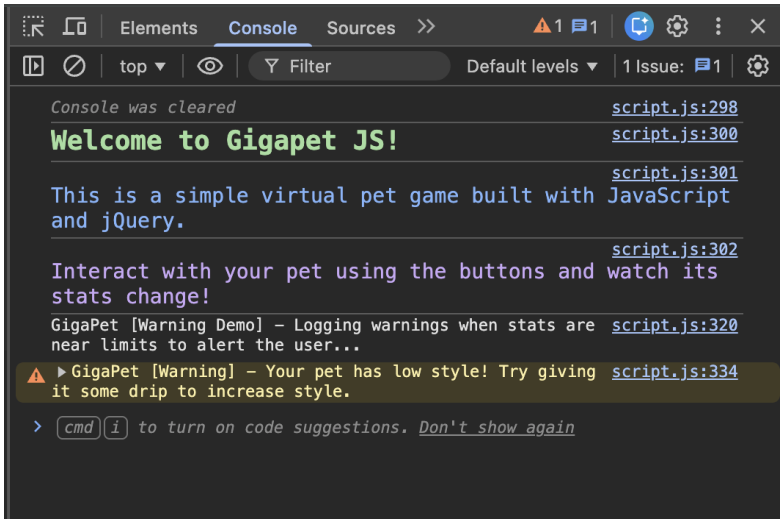
It uses console.info() to make a log in the console at Info level and displays the pet_info object. This function is triggered by clicking the **Log Info** button.

Screenshot of Console after clicking the **Log Info** button.

1.2 Log Warning

Log Warning Message implementation is done through the *demoLogWarning()* function.

It uses console.warn() to make a log in the console at Warnings level and displays a warning icon, highlighting the message yellow. This function is triggered by clicking the **Log Warning** button and checks for the approaching limits of weight (<= 2 or >= 8), happiness (<= 2) and style (<=2 or >= 8), logging any applicable warning messages to the console. If limits are not being approached and therefore no warning messages are applicable, it will just show a general log that proves the function was triggered.

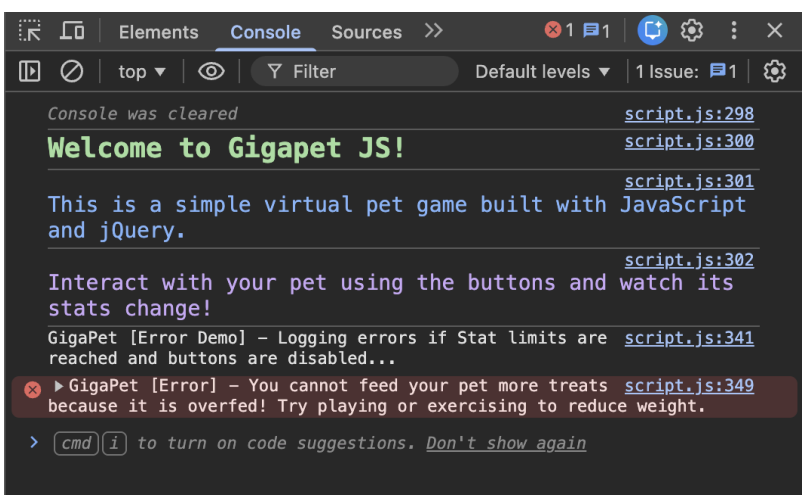


Screenshot of Console after clicking the **Log Warning** button (at Style = 2).

1.3 Log Error

Log Error Message implementation is done through the *demoLogError()* function.

It uses console.error() to make a log in the console at Errors level and displays a crossed circle icon, highlighting the message red. This function is triggered by clicking the **Log Error** button and checks for the limits of weight (0 or 10), happiness (10) and style (0 or 10), logging any applicable error message to the console. If limits are not hit and therefore no error messages are applicable, it will just show a general log that proves the function was triggered.

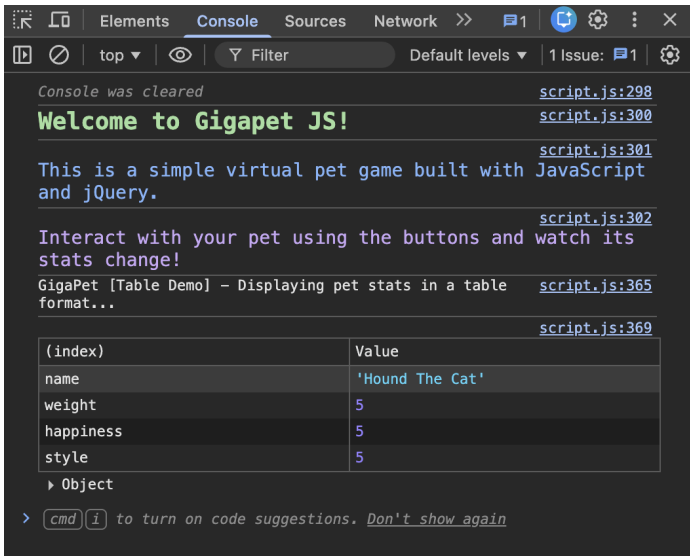


Screenshot of Console after clicking the **Log Error** button (at Weight = 10).

1.4 Log Table

Log Table Message implementation is done through the *demoLogTable()* function.

It uses console.table() to render the pet_info array of objects (consisting of name, weight, happiness and style) as a sortable, formatted table. This function is triggered by clicking the **Log Table** button.

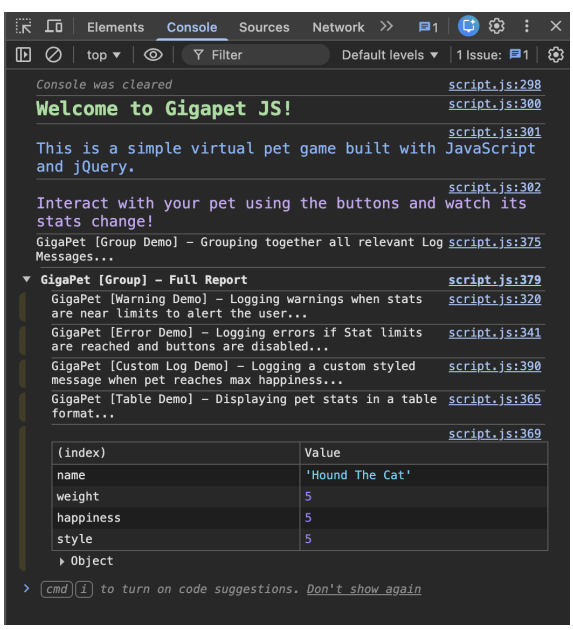


Screenshot of Console after clicking the **Log Table** button.

1.5 Log Group

Log Group Message implementation is done through the *demoLogGroup()* function.

It uses console.group() to create a collapsible label in the console until it reaches console.groupEnd(). This function is triggered by clicking the **Log Group** button and gives a full report of the current pet_info by triggering all the previous Log Message implementations inside of the collapsible.

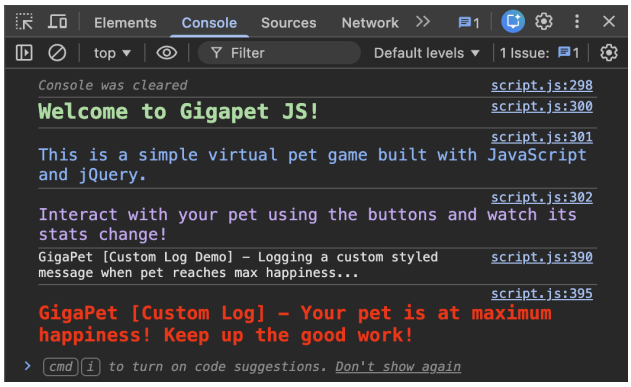


Screenshot of Console after clicking the **Log Group** button. (No stat limits reached)

1.6 Log Custom

Log Custom Message implementation is done through the *demoLogCustom()* function.

It uses console.log() and a conditional statement to display a Custom Log Message when Happiness is at max. This function is triggered by clicking the **Log Custom** button. If Happiness is not at max therefore not triggering the conditional it will just show a general log that proves the function was triggered.



Screenshot of Console after clicking the **Log Custom** button (at Happiness = 10).

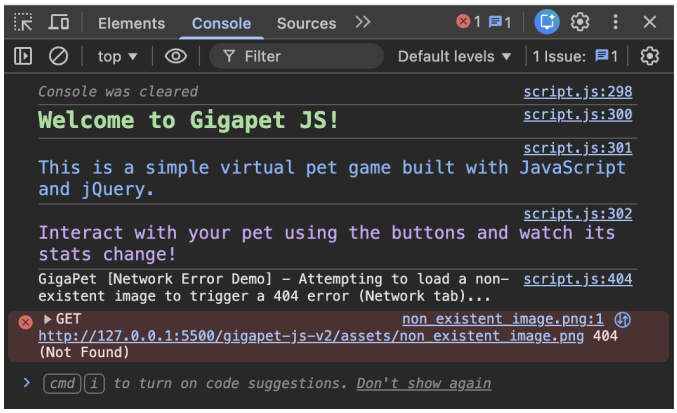
2 View messages logged by the browser

The buttons and relevant instructions mentioned below are all under the **View messages logged by the browser** section of the DevTools Demo Panel.

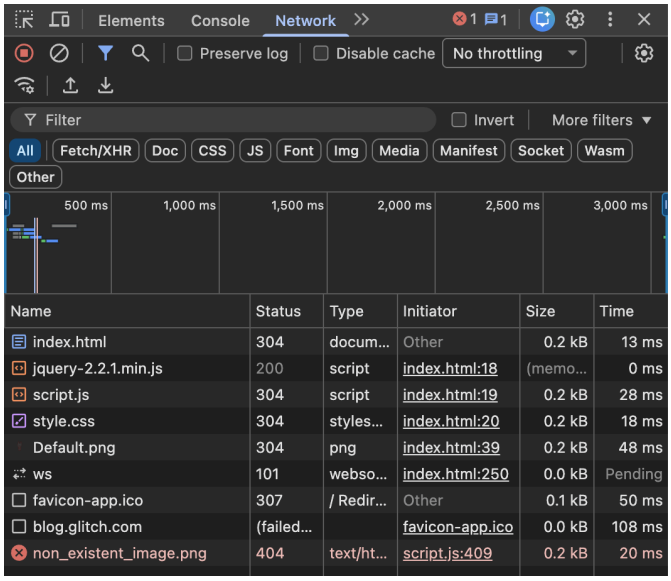
2.1 Cause 404 network error

404 Error Browser Log Message implementation is done through the *demoCause404()* function.

It simulates a network error by trying to load a non-existent image, causing the browser to log a 404 error in the console, which can also be found in the Network Panel of the DevTools. This function is triggered by clicking the **Cause 404 Error** button.



Screenshot of Console after clicking the **Cause 404 Error** button.

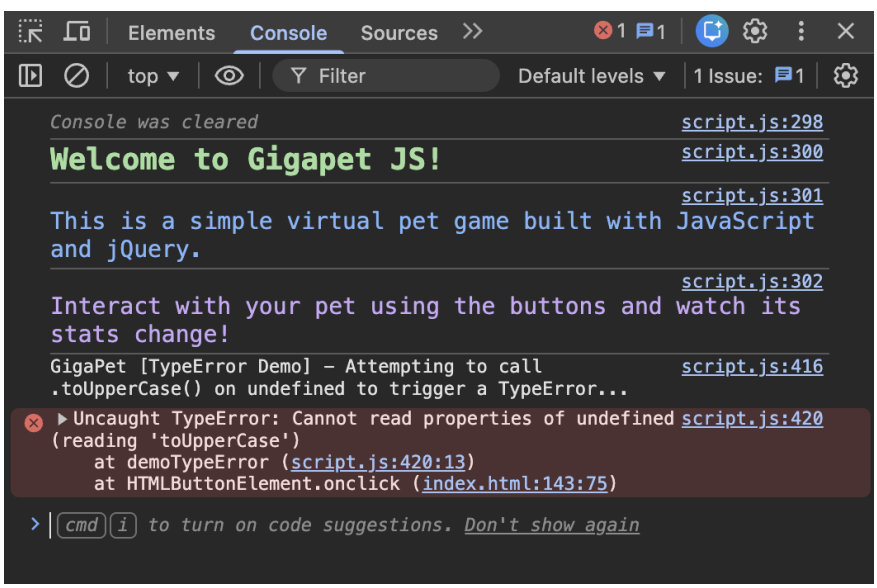


Screenshot of Network Panel after clicking the **Cause 404 Error** button.

2.2 Cause TypeError

TypeError Browser Log Message implementation is done through the *demoTypeError()* function.

It simulates a TypeError by trying to call .toUpperCase() on an undefined value, causing the browser to log an Uncaught TypeError in the console. This function is triggered by clicking the **Cause TypeError** button.

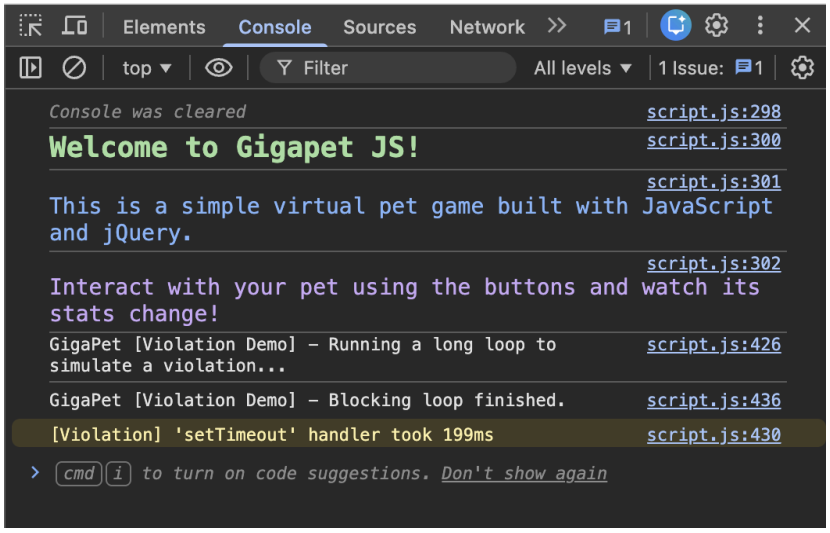


Screenshot of Console after clicking the **Cause TypeError** button.

2.3 Cause Violation

Violation Browser Log Message implementation is done through the *demoViolation()* function.

It simulates a Violation by running a long loop on the main thread, causing the browser to log a Violation in the console. The console message is at Verbose level so the Verbose level filter must be on for it to appear. This function is triggered by clicking the **Cause Violation** button.

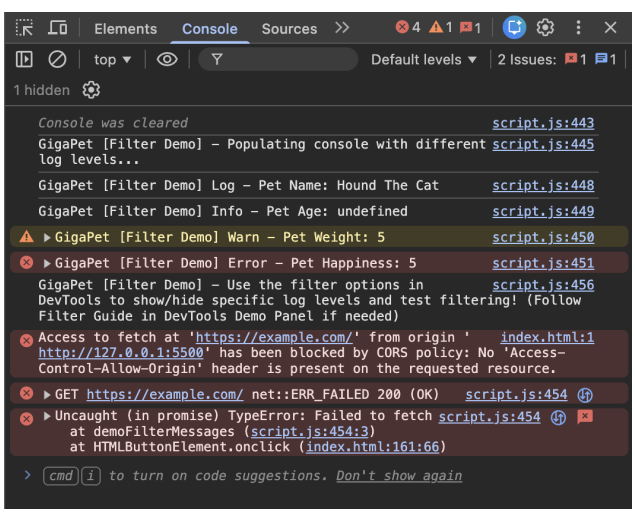


Screenshot of Console after clicking the **Cause Violation** button.

3 Filter Messages

The buttons and relevant instructions mentioned below are all under the **Filter Messages** section of the DevTools Demo Panel.

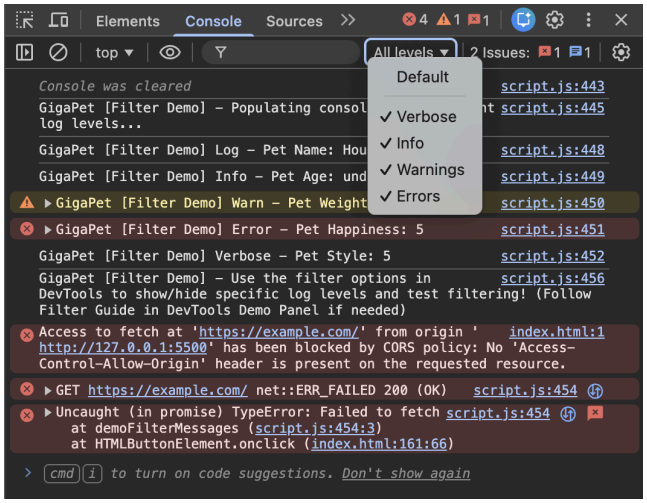
Filter Message implementation is done through instructions provided in said section, as well as the *demoFilterMessages()* function. Said function logs one message at each level filter (Info, Warning, Error, Verbose) through the use of console.info(), console.warn(), console.error() and console.debugg() respectively. It also uses fetch('https://example.com') to trigger a browser-generated log message (which will not show up under user messages). This function can be triggered by clicking the **Populate Filters** button.



Screenshot of Console after clicking the **Populate Filters** button. (Default filters)

3.1 Filter by log level

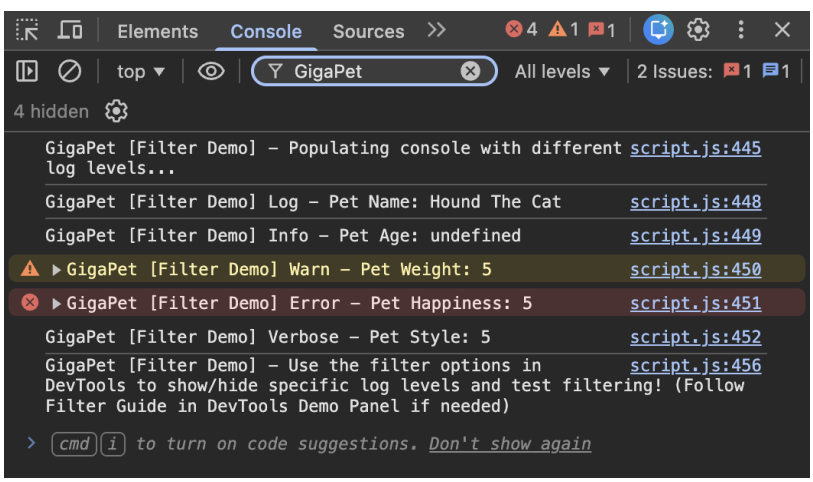
Filtering by log level can be done through the “Default levels” (level filters) dropdown in the Console toolbar.



Screenshot of Console with level filters dropdown open after turning on Verbose level filter.

3.2 Filter by text

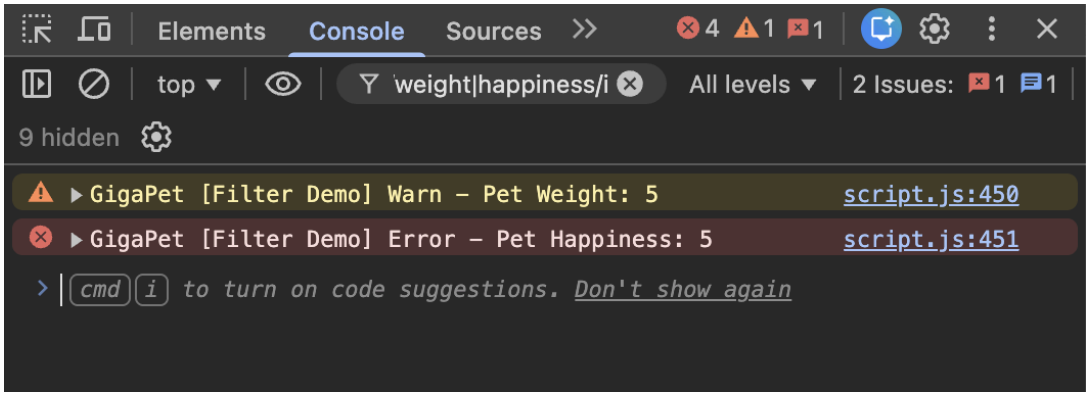
Filtering by text is done by entering text into the filter bar to only show messages containing that text.



Screenshot of Console after entering “GigaPet” into the filter bar to filter for messages containing that text.

3.4 Filter by regular expression

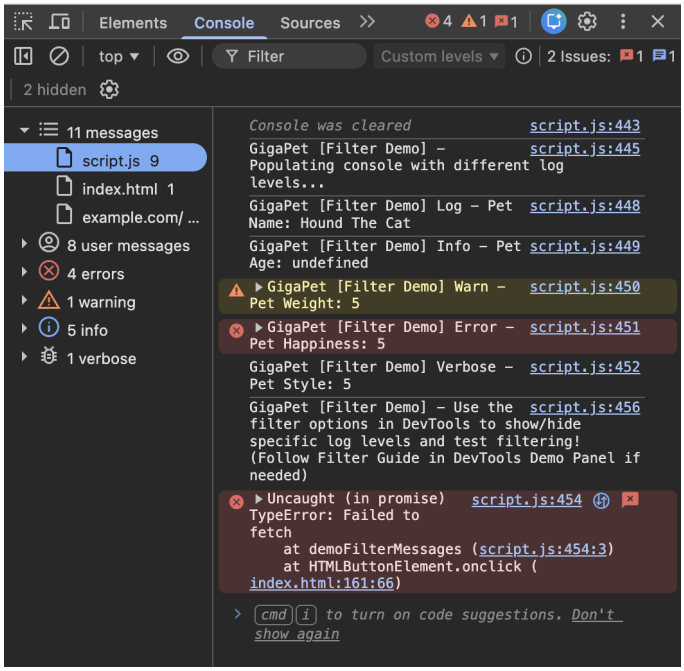
Filtering by regular expressions can be done by entering regular expressions into the filter bar to show messages containing the specified terms.



Screenshot of Console after entering the regular expression “/weight|happiness/i” into the filter bar to filter for messages containing those terms.

3.5 Filter by message source

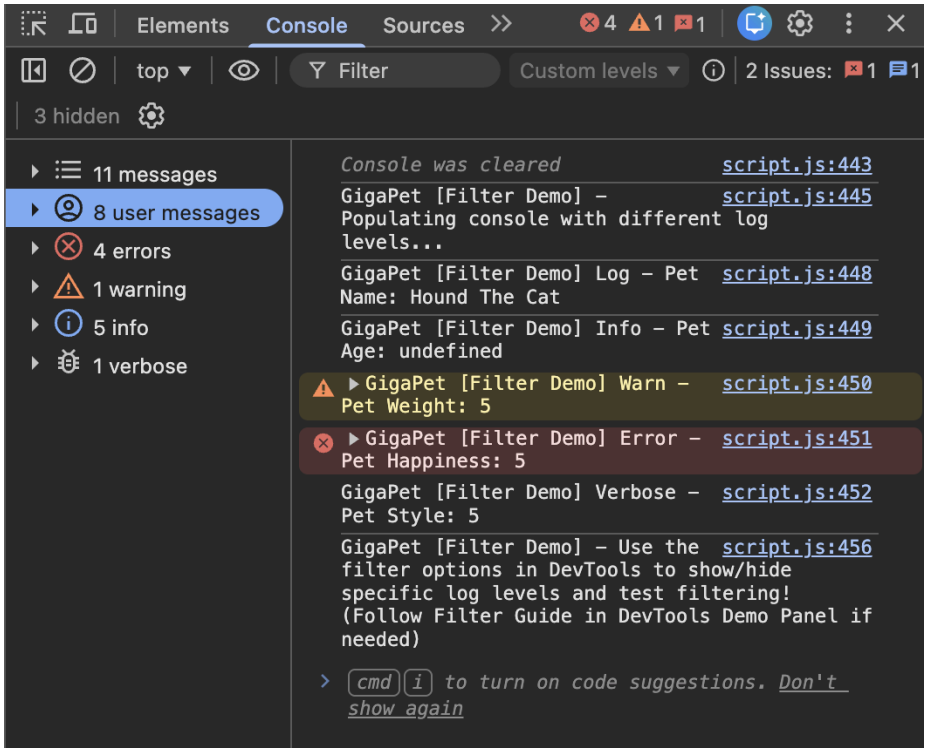
Filtering by message source can be done by using the sidebar and selecting a File Name in the Messages dropdown.



Screenshot of Console with sidebar opened and filtering by script.js for messages originating from that source.

3.6 Filter by user messages

Filtering by user messages can be done by using the sidebar and selecting user messages.

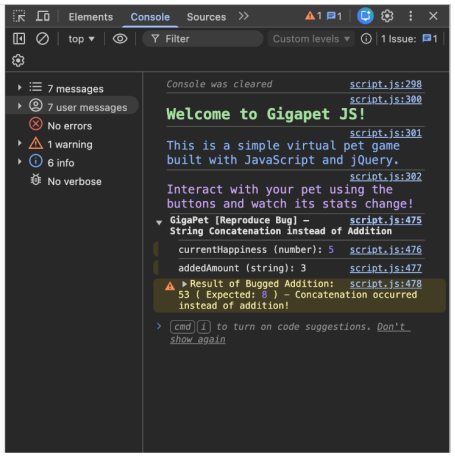


Screenshot of Console with sidebar open and filtering by user messages.

4 Reproduce a bug

The buttons and relevant instructions mentioned below are all under the **Reproduce a Bug & Apply a Fix** section of the DevTools Demo Panel.

Reproducing a bug implementation is done through the *buggedAddition()* and *demoReproduceBug()* functions. The buggedAddition() function is a Helper function that concatenates values of happiness and added value together instead of adding them, if one or both values is a string. It is called inside of the demoReproduceBug() function, which passes the current happiness value and a string for the added amount. It logs a Warning message in the console and injects a text demonstrating the bugged result into the inner HTML. This function can be triggered by clicking the **Reproduce Bug** button.

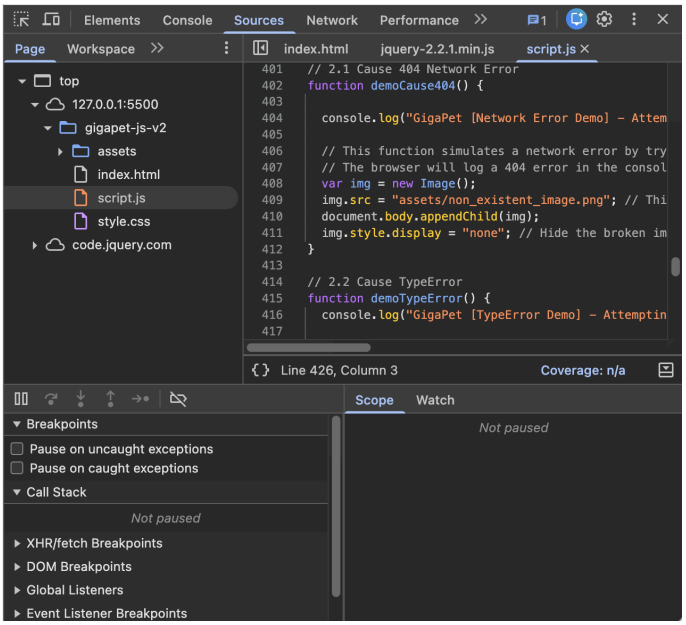


Screenshot of the Webpage and Console after clicking the **Reproduce Bug** button.

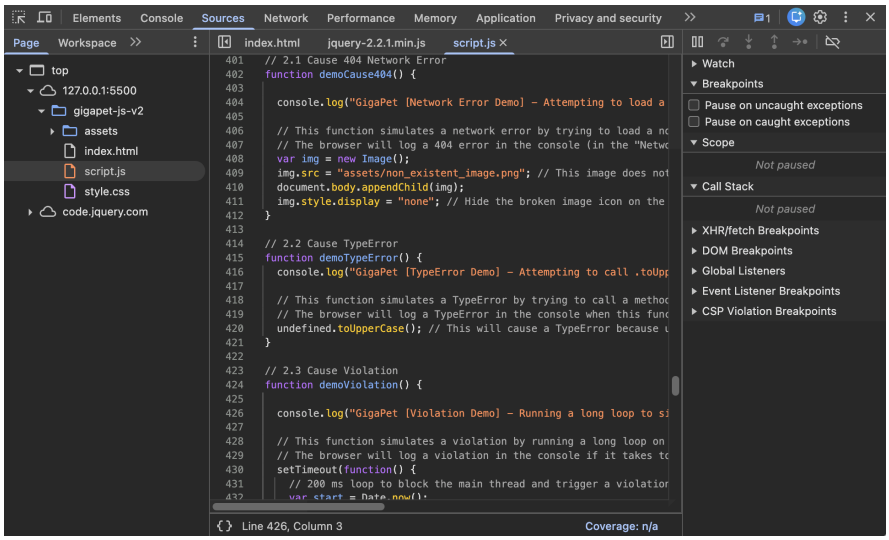
5 Get Familiar with the Sources UI

Getting Familiar with the Sources UI implementation is done through instructions in the **Sources, Breakpoints & Check Variables** section of the DevTools Demo Panel.

Switching to the Sources Panel in the DevTools, the panel is divided into three main sections: the Page tab with the file tree on the left, the Code Editor on the right (or the middle if the DevTools is stretched long), and the Debugger below them both (or to the rightmost of them if the DevTools is stretched long), which is where you can find the Breakpoints, Scope and Watch.



Screenshot of Sources Panel of DevTools.

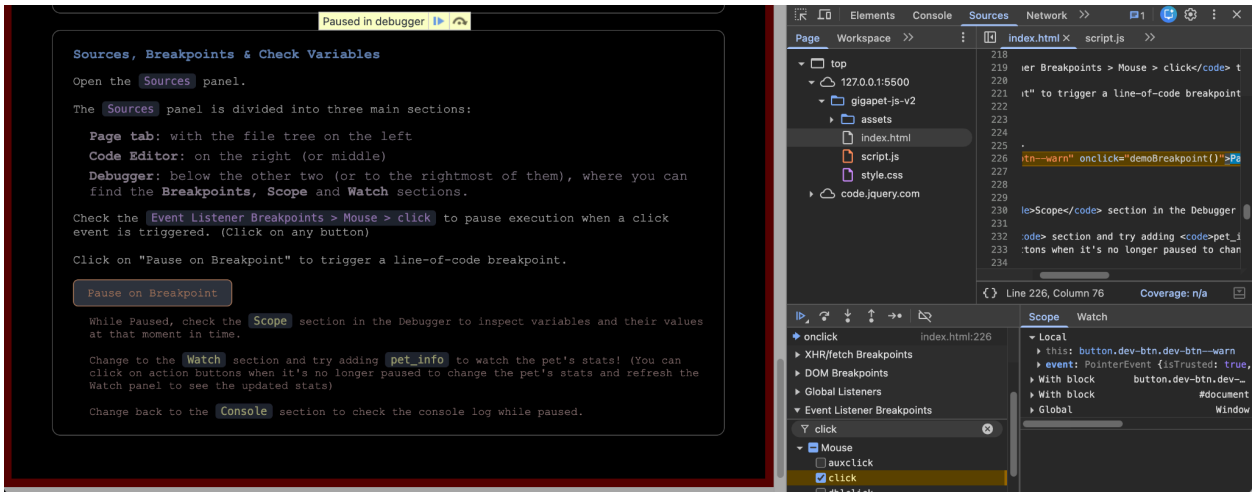


Screenshot of Sources Panel of DevTools stretched long.

6 Pause the code with a breakpoint

Pausing the code with a breakpoint implementation is done through instructions in the **Sources, Breakpoints & Check Variables** section of the DevTools Demo Panel.

You can use the Debugger section of the Sources Panel to pause the debugger through checking different breakpoints. The instructions on the page tell you to turn on the Event Listener Breakpoint on Mouse clicks which will cause the debugger to pause as soon as you interact with a click event (click on any button in the page).

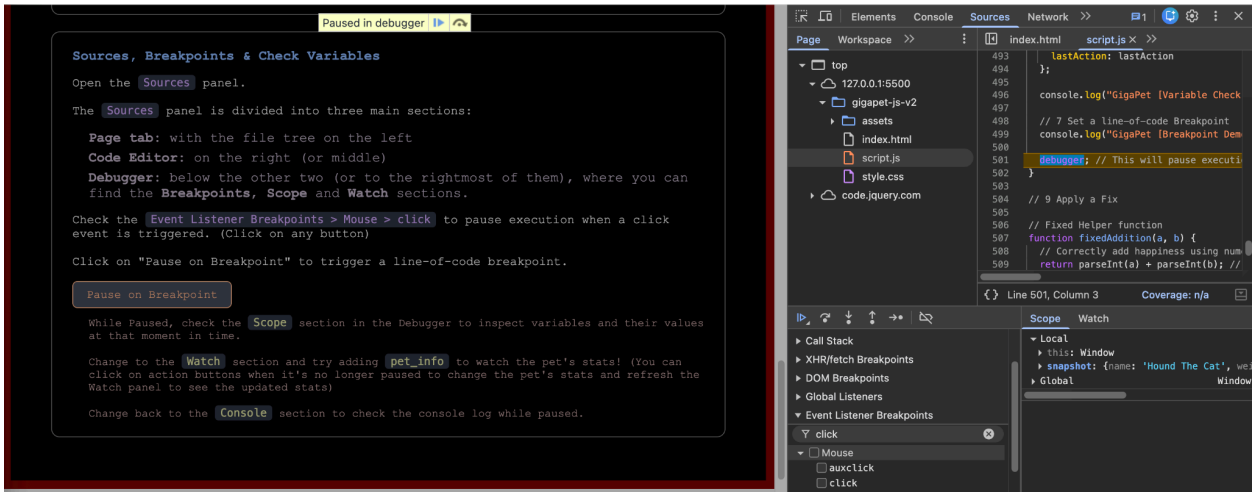


Screenshot of paused webpage and DevTools after clicking on a button while Event Listener Breakpoint mouse click settings is turned on.

7 Set a line-of-code breakpoint

The relevant buttons and instructions mentioned below are under the **Sources, Breakpoints & Check Variables** section of the DevTools Demo Panel.

Setting a line-of-code breakpoint implementation is done through the *demoBreakpoint()* function, which pauses execution in DevTools through a “debugger;” line in the code. This function is triggered by clicking the **Pause the Breakpoint** button.



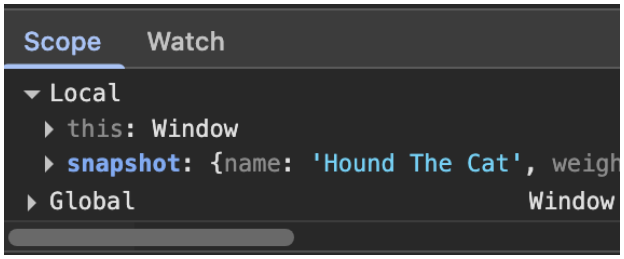
Screenshot of paused webpage and DevTools after clicking on the **Pause the Breakpoint** button. (Note how the click breakpoint is not turned on and the debugger one liner in the code is highlighted)

8 Check variable values

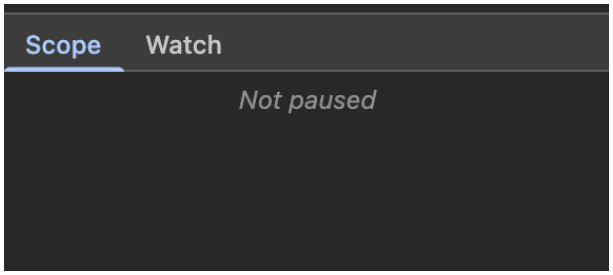
The relevant instructions mentioned below are under the **Sources, Breakpoints & Check Variables** section of the DevTools Demo Panel.

8.1 The Scope Pane

Scope Pane variable inspection can be done through the Scope Pane on the Debugger, where you can inspect all the variables and their values at the moment in time the debugger was paused. The Scope Pane can't be used to inspect variables without pausing the debugger first.



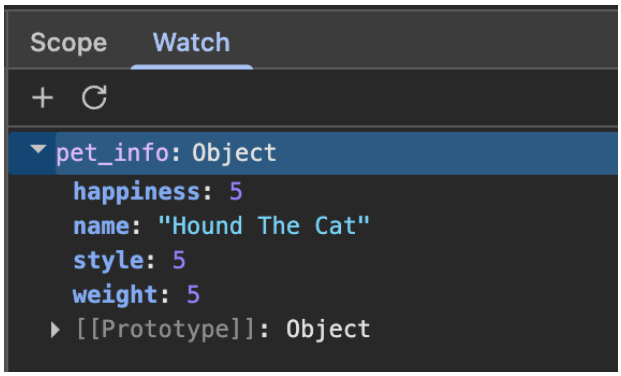
Screenshot of the Scope Panel while the Debugger is paused.



Screenshot of the Scope Pane while the Debugger is NOT paused. (Inspection can't be done)

8.2 Watch Expressions

Watch expressions variable inspection can be done through the Watch Pane on the Debugger, where you can add a Watch expression by clicking on plus symbol and entering a variable you want to watch. Unlike the Scope Pane the Watch Pane can be used to inspect variables even if the debugger isn't paused, but to see the updated values after they change you need to click on the refresh symbol first.



Screenshot of the Watch Pane after adding the pet_info watch expression.

8.3 The Console

Variables inspect on the console is implemented through the demoBreakpoint() function which besides pausing the debugger with a line-of-code breakpoint also logs a snapshot of the pet_info variables and lastAction. Which you can see by switching back to the Console to check the console log while paused. You can also enter or just type a variable name into the console in order to check its value(s).

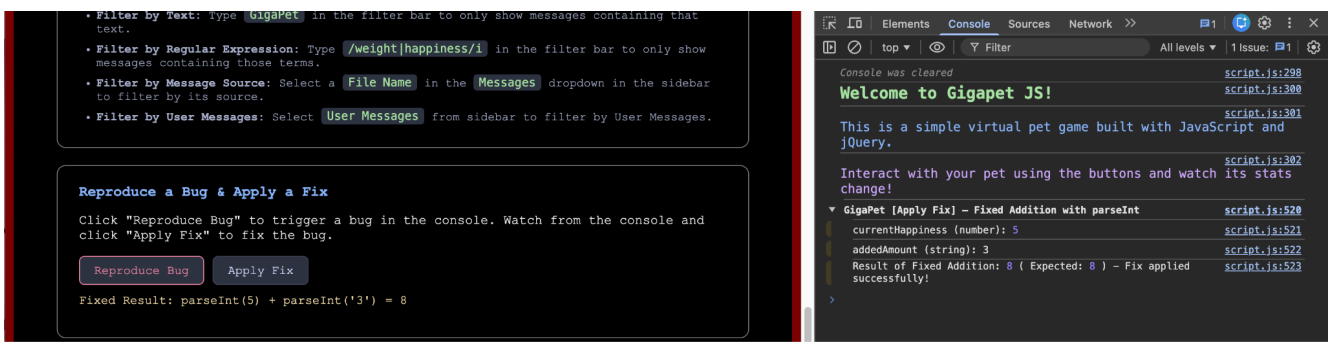
Screenshot of Console after clicking the **Pause on Breakpoint** button.

Screenshot of typing “pet_info” variable name into the console to check its values.

9 Apply a fix

The buttons and relevant instructions mentioned below are all under the **Reproduce a Bug & Apply a Fix** section of the DevTools Demo Panel.

Applying a fiximplementation is done through the *fixedAddition()* and *demoApplyFix()* functions.The fixedAddition() function is a Helper function that ensures the variables are read as integers by applying parseInt() on both variables passed through in order to perform a proper addition instead of a concatenation. It is called inside of the demoApplyFix() function, which passes the current happiness value and a string for the added amount. It logs a message in the console and injects a text demonstrating the fixed result into the inner HTML. This function can be triggered by clicking the **Apply Fix** button.



Screenshot of the Webpage and Console after clicking the **Apply Fix** button.